Mojo 🔥

in the LLVM Ecosystem

LLVM Dev Meeting 2026 · Mojo Workshop

Modular, a Qualcomm company

Billy Zhu · Lukas Hermann · Weiwei Chen

{billyzhu, lukas, weiwei.chen}@modular.com

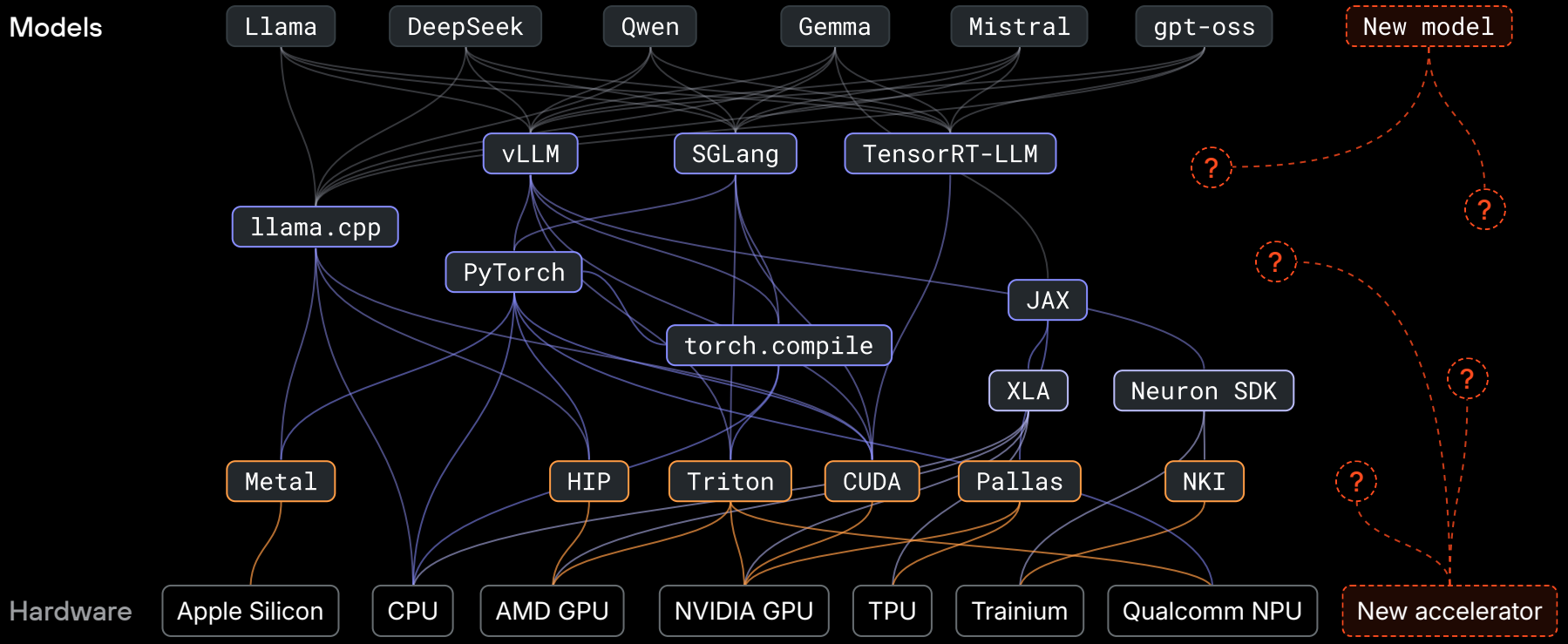
Why Mojo?

The problem it was built to solve, and why a new language was the answer

Modular's goal

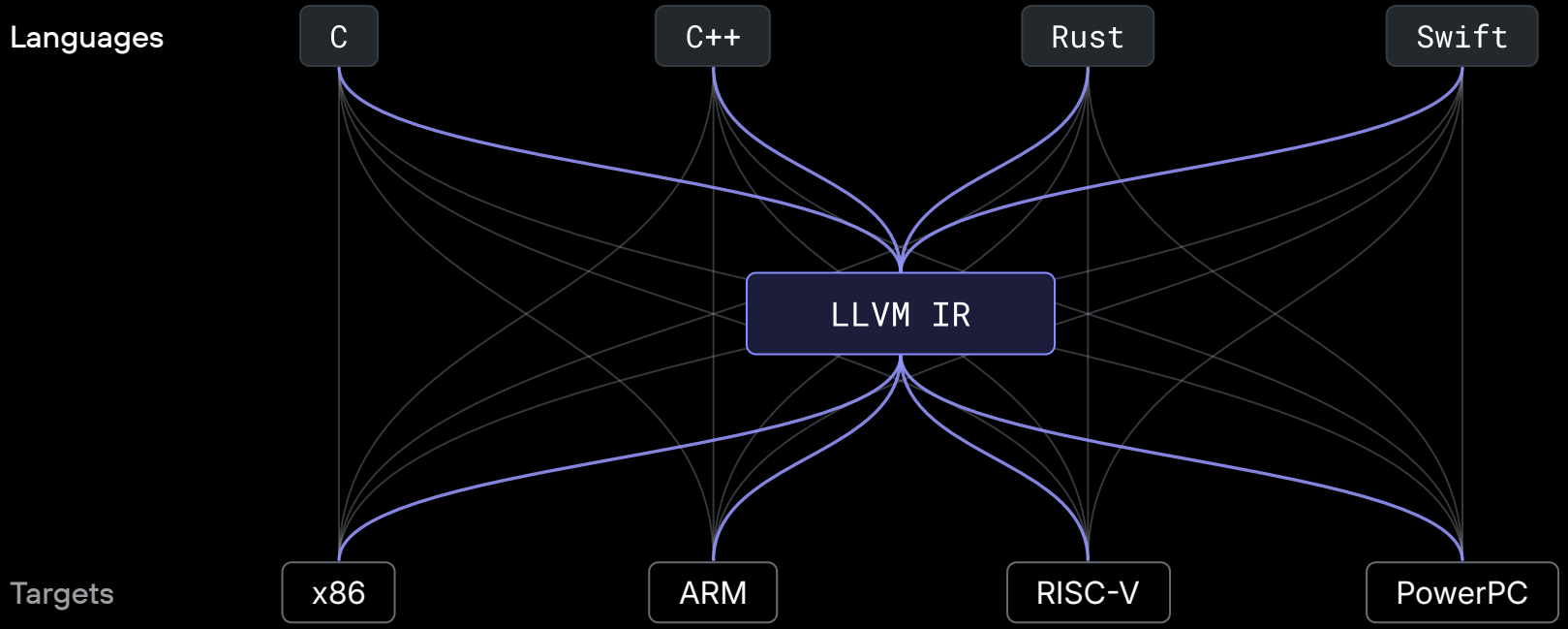
A unified inference stack that runs on heterogeneous hardware.

Why is this hard?



Incentives reward point solutions. The result is a web of fragmented systems.

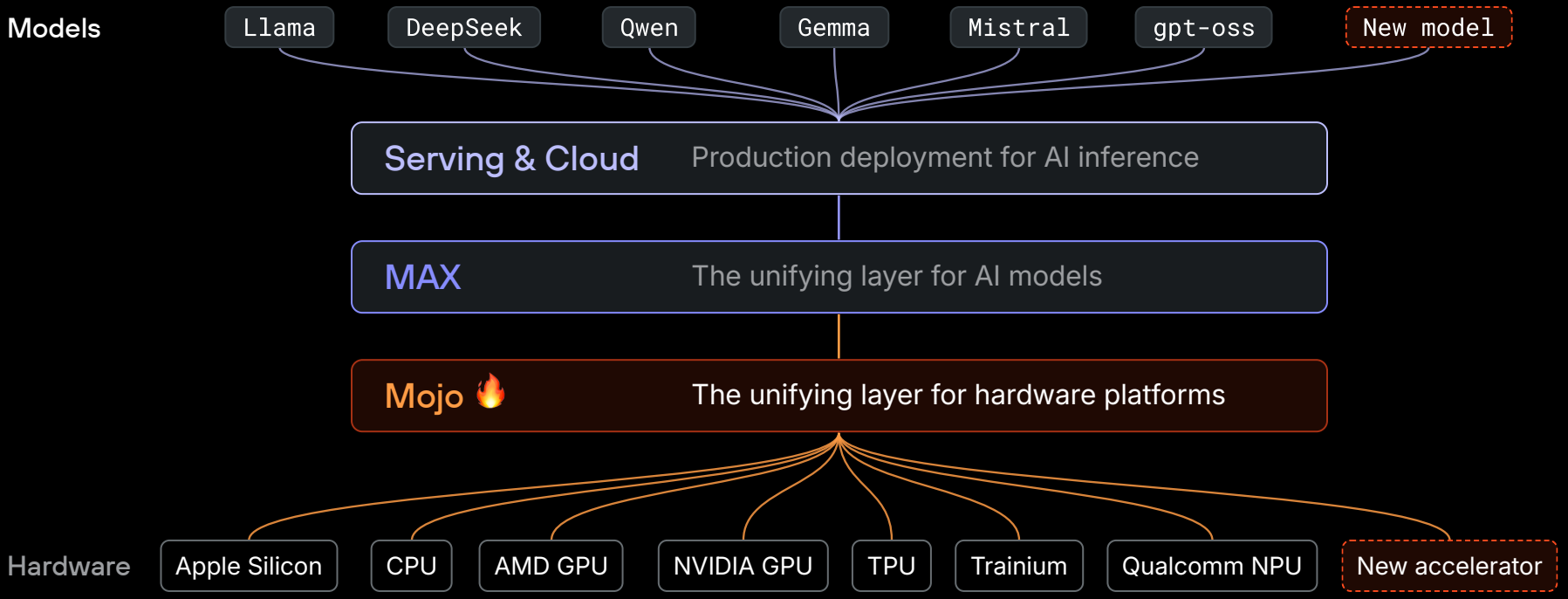
The same shape of problem LLVM solved



`m × n` Without a shared IR

`m + n` With LLVM in the middle

Mojo unifies heterogeneous compute



Objective

Maximum performance on every hardware platform

Which means bridging different programming models.

Traditional compute

LLVM IR

x86 ARM RISC-V PowerPC

Heterogeneous accelerators

No single programming model covers them all

GPU TPU NPU Trainium …

Every hardware ecosystem ships its own kernel language

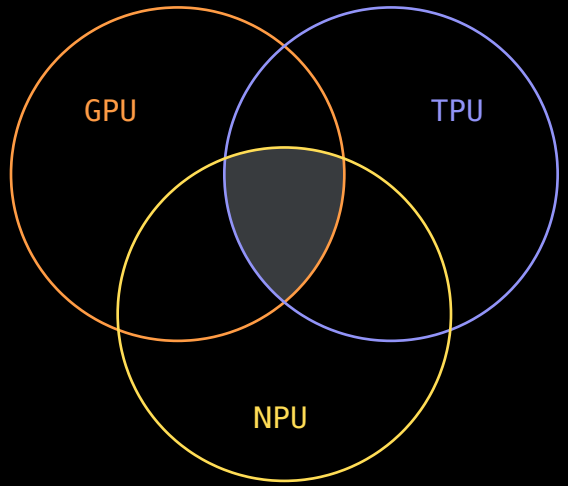
NVIDIA	AMD	Google TPU	AWS Trainium
`CUDA C++ / CuTe`	`HIP`	`Pallas`	`NKI`

A dual-sided moat: hard to leave, hard to enter.

This reinforces fragmentation.

How do we unify this?

What doesn't work: the lowest common denominator



Uniformity at the expense of performance.
Target-specific capabilities stay hidden.

Getting performance back takes specialized knowledge:

Smart compilers

Find optimizations from each hardware's characteristics

Vendor libraries

Optimized low-level libraries, per hardware

Every new operation needs a specialist

Smart compilers

A compiler pattern for every transformation

Vendor libraries

A target-specific implementation of every op

Key unlock

Performance knowledge must be programmable by domain experts.

Not sealed inside a compiler or a vendor library.

Modular's approach

A sufficiently ~~smart compiler~~ expressive language

Each hardware platform thrives via specialization, not a uniform mold.

Hardware knowledge lives in user libraries, not vendor binaries.

MLIR provides half of the foundation

The common foundation for modern AI compilers

`IREE · Triton · torch-mlir · OpenXLA · tt-mlir · mlir-aie`

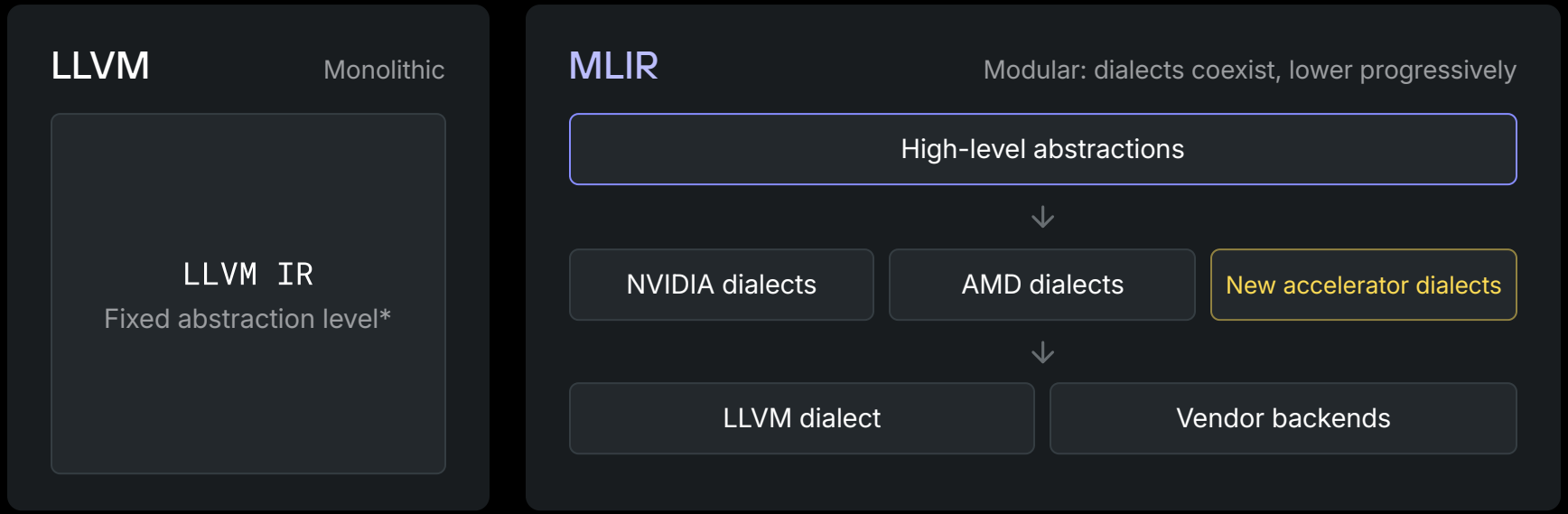
?

MLIR

`llvm dialect`
The LLVM programming model

`accelerator dialects`
Accelerators already use MLIR

Why MLIR worked where LLVM fell short



The natural substrate for unifying heterogeneous compute.

The missing half: Mojo, the user-facing layer

An opinionated, transparent wrapper over MLIR

Mojo 🔥

- Meta-programming over MLIR
  - Performant portability
- Safety, composition, diagnostics, tooling
  - Ease of use of a modern language

MLIR Mojo bottoms out here, not at a fixed set of builtin primitives.

The new landscape

01
A new accelerator

→

02
Exposes native ops as MLIR dialects, or via LLVM

→

03
Builds safe abstractions in Mojo libraries

→

04
Users keep one language and one toolchain

The problem

Uniform abstractions sacrifice target performance.

Compiler-owned intelligence doesn't scale organizationally.

The answer

MLIR lets target-specific abstractions coexist.

Mojo lets programmers build and compose them safely.

Up next

01 **Language**
The tools Mojo gives programmers

02 **Libraries**
Where hardware abstractions live

03 **Compiler**
How Mojo reaches MLIR, LLVM, and devices

Mojo in five minutes

A taste of the ideas that make Mojo distinctive

Python inherited syntax

```
struct Person:
    var name: String
    var favorite_number: Int32
    var date_of_birth: Tuple[Int, Int, Int]

def is_billy(p: Person) -> Bool:
    return p.name == "Billy"
```

Traits

```
trait Greetable:
    def greet(self):
        ...

struct Person(Greetable):
    var name: String
    var favorite_number: Int32
    var date_of_birth: Tuple[Int, Int, Int]

    def greet(self):
        print(t"Hello, {self.name}")

def use_greetable(g: Some[Greetable]):
    g.greet()
```

Mutable value semantics

Mojo

```
def by_immutable_reference(p: Person): ...
def by_mutable_reference(mut p: Person): ...
def parametrically_mutable_reference(ref p: Person): ...
def by_value(var p: Person): ...
```

C++

```
void by_immutable_reference(const Person& p) {}
void by_mutable_reference(Person& p) {}
template <typename P>
    requires std::same_as<std::remove_const_t<P>, Person>
void parametrically_mutable_reference(P& p) {}
void by_value(Person p) {}
```

Comptime: compile-time evaluation

```mojo
def sum(l: List[Int]) -> Int:
    var result = 0
    for elt in l:
        result += elt
    return result

# Does not allocate
def print_sum():
    comptime s = sum([1, 2, 3])
    print(s) # 6
    comptime s2 = sum([1, 2, 3, 4, 5])
    print(s2) # 15
```

```mojo
# branching disappears at runtime
def conditional():
    comptime s = sum([0, 0, 0])
    comptime if s == 0:
        print("zilch")
    else:
        print(s)
```

compiles to

```mojo
# what ships in the binary
def conditional():
    print("zilch")
```

Origins: Mojo's borrow checker

```
def mutate(mut value: List[Int], mut value2: List[Int]):
    pass

def main():
    var l: List[Int] = [1, 2, 3]
    mutate(l, l)
    # error: aliasing values passed mutably to 'value' argument
    # and passed mutably to 'value2' argument in 'mutate' call
```

Parameters: generic programming

```
# Types and values can be parameters
struct Array[T: AnyType, length: Int]:
    # ...
    def __contains__(
        self, value: Self.T
    ) -> Bool where conforms_to(Self.T, Equatable):
        comptime for i in range(Self.length):
            if self[i] == value:
                return True
        return False

def contains_5(l: Array[Int, _]) -> Bool:
    return 5 in l
```

Library-defined abstractions

Where Mojo's core abstractions are defined, in Mojo itself

Every primitive lives in libraries

```
struct SIMD[dtype: DType, length: SIMDLength]:
    # ...

comptime Scalar = SIMD[
    _, length=__mlir_attr[`#lit.struct<{_mlir_value = 1}> : `, SIMDLength]
]

comptime Int = Scalar[DType.int]
comptime Int8 = Scalar[DType.int8]
comptime UInt8 = Scalar[DType.uint8]
comptime Int16 = Scalar[DType.int16]
comptime UInt16 = Scalar[DType.uint16]
comptime Int32 = Scalar[DType.int32]
comptime UInt32 = Scalar[DType.uint32]
comptime Int64 = Scalar[DType.int64]
comptime UInt64 = Scalar[DType.uint64]
comptime Int128 = Scalar[DType.int128]
comptime UInt128 = Scalar[DType.uint128]
comptime Int256 = Scalar[DType.int256]
comptime UInt256 = Scalar[DType.uint256]
```

LLVM composition

```mojo
@inline(.always)
def iota[
    dtype: DType, width: Int
](offset: Scalar[dtype] = Scalar[dtype](0)) -> SIMD[dtype, width]:

    comptime step_dtype = dtype if dtype.is_integral() else DType.int

    var step: SIMD[step_dtype, width]
    if __is_run_in_comptime_interpreter:
        # ...
    else:
        # you can just call llvm inline!!
        step = llvm_intrinsic[
            "llvm.stepvector", SIMD[step_dtype, width], has_side_effect=False
        ]()
    return step.cast[dtype]() + offset
```

TileTensor overview (simplified)

```
struct TileTensor[
    mut: Bool,
    //,
    dtype: DType,
1   LayoutType: TensorLayout,
    origin: Origin[mut=mut],
    *,
    address_space: AddressSpace,
2   Engine: TensorEngine
]:
    var _storage: Self.Engine.StorageType[
        Self.dtype, Self.origin, Self.address_space
    ]
    var layout: Self.LayoutType
```

1. **`LayoutType`**

   Describes the static and dynamic mapping between coordinates and the underlying storage.

2. **`Engine`**

   Describes how operations between tensors should be lowered for a given hardware target.

Coord

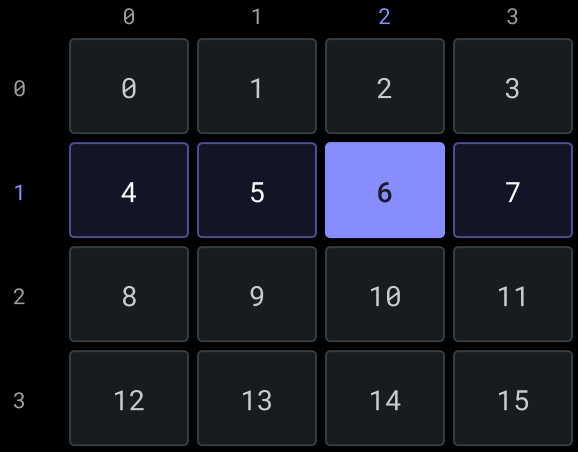
```
from layout import Coord, Idx, coord
from std.sys import size_of

def main():
    var fully_dynamic = Coord(Int32(128), Int32(8))
    var semi_dynamic = Coord(Idx[128], Int32(8))
    var fully_static = coord[128, 8]
    var nested = Coord(Coord(Int32(64), Int32(2)), Int32(8))

    print(size_of[type_of(nested)]())        # 12
    print(size_of[type_of(fully_dynamic)]()) # 8
    print(size_of[type_of(semi_dynamic)]())  # 4
    print(size_of[type_of(fully_static)]())  # 0
```

- Nested tuples that describe the mapping of coordinates to indices.
- Can combine `comptime` and dynamic information.
- Can also represent mixtures of dtype.
- ABI stores exactly what it needs.

Layout

```mojo
from layout import Coord, Idx, row_major

def main():
    var lyt = row_major[4, 4]()
    # shape = (4, 4), stride = (4, 1)
    print(lyt(Coord(1, 2)))  # 6
```

- Represents a shape and stride coordinate mapping using layout algebra.
- Allows crouton-based or row-major mapping without requiring users to change how they think about indexing.



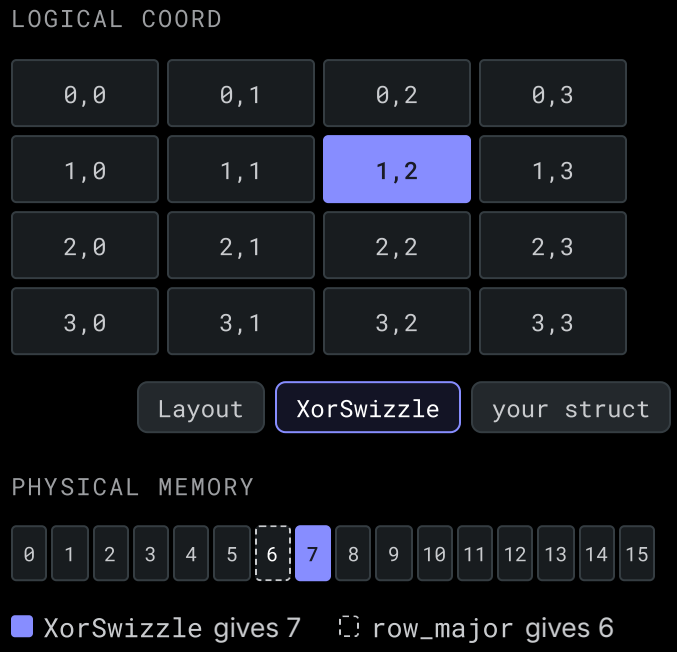
coord (1, 2)
stride (4, 1)

index 1·4 + 2·1 = 6

TensorLayout trait

```
trait TensorLayout:
    def __call__(self, c: Coord) -> Int: ...
    # logical coord → physical offset

struct XorSwizzle(TensorLayout):
    def __call__(self, c: Coord) -> Int:
        return c[0] * 4 + (c[1] ^ c[0])

var t = TileTensor(ptr, XorSwizzle())
print(t.layout(Coord(1, 2)))  # 7
```

- Anything that conforms to `TensorLayout` can be used as a layout.
- Arbitrary mappings from logical layout to physical memory access.



TensorEngine

- Contains the non-owning storage type.
- Specifies how to map operations between tensors to hardware instructions.
- Lets different hardware use its own optimizations and paradigms (e.g. SIMD vs. a single MLIR op).

```
trait TensorEngine:
    comptime StorageType[
        mut: Bool,
        //,
        dtype: DType,
        origin: Origin[mut=mut],
        address_space: AddressSpace,
    ]: TrivialRegisterPassable

    @staticmethod
    def iadd[
        # ...
    ](
        storage: Tuple[
            Self.StorageType[
                dtype, self_origin, self_address_space
            ],
            SelfLayoutType,
        ],
        other: Tuple[
            OtherEngine.StorageType[
                dtype, other_origin, other_address_space
            ],
            OtherLayoutType,
        ],
    ):
        __mlir_op.`my_dialect.add`(storage[0], other[0])
```

Tile-based vec_add

```
def vec_add(
    var c: MutTileTensor,
    a: ImmTileTensor[dtype=c.dtype, Engine=c.Engine, ...],
    b: ImmTileTensor[dtype=c.dtype, Engine=c.Engine, ...],
):
    comptime assert conforms_to(c.Engine, TensorOps)
    c += a + b
```

Layout

Uses layout compatibility to send the indexing information.

Engine

Uses the engine to describe how to lower the operation.

Full Power of MLIR & LLVM

Mojo exposes the underlying IR to language users

Mojo exposes the full power of MLIR/LLVM

```mojo
from std.sys import is_nvidia_gpu, llvm_intrinsic
from std.sys.info import _cdna_4_or_newer, _is_sm_100x_or_newer

from max.gpu import global_idx


def to_fp4x2[dtype: DType, //](pair: SIMD[dtype, 2]) -> UInt8:
    comptime assert dtype.is_floating_point(), "expects a pair of floats"
    var a = pair[0].cast[.float32]()
    var b = pair[1].cast[.float32]()

    comptime if is_nvidia_gpu():
        comptime assert _is_sm_100x_or_newer(), "FP4 needs Blackwell (sm_100a)"
        return llvm_intrinsic[
            "llvm.nvvm.ff.to.e2m1x2.rn.satfinite", UInt16, has_side_effect=False
        ](a, b, False).cast[.uint8]()
    elif is_amd_gpu():
        comptime assert _cdna_4_or_newer(), "FP4 needs CDNA 4 (gfx950)"
        return llvm_intrinsic[
            "llvm.amdgcn.cvt.scalef32.pk.fp4.f32", Int32, has_side_effect=False
        ](Int32(0), a, b, Float32(1.0), Int32(0)).cast[.uint8]()
    else:
        comptime assert False, "FP4 packing needs an NVIDIA or AMD GPU"


def fp4_kernel[
    dtype: DType
](
    src: ImmPointer[SIMD[dtype, 2], ImmutAnyOrigin],
    dst: MutPointer[UInt8, MutAnyOrigin],
):
    var i = global_idx.x
    dst[unsafe_offset=i] = to_fp4x2(src[unsafe_offset=i])
```

- No compiler work to reach new hardware features
- One mechanism for every target
- Integrated with the Mojo type system
- Built in with the stdlib

Mojo exposes the full power of MLIR/LLVM

Types

Name any MLIR type

`__mlir_type` lets a struct wrap an IR type directly; `Bool` is a wrapper around `!kgen.scalar<bool>`.

Operations

Utilize any MLIR construct

`__mlir_op` and `__mlir_attr` reach the IR from library code.

Cost

Wrappers compile away

Zero cost abstraction, no runtime overhead.

`Bool` from the Mojo standard library (simplified)

```
struct Bool(Boolable, Comparable, Hashable, Writable, ...):
    # Provide storage as an MLIR i1
    var _value: __mlir_type.`!kgen.scalar<bool>`

    # Enable "Bool.true" as a literal and for enums.
    comptime true = Bool(mlir_value=__mlir_attr.true)

    # a = ~b
    def __invert__(self) -> Bool:
        return __mlir_op.`pop.xor`(
            self._value, __mlir_attr.true
        )

    # c = a & b
    def __and__(self, rhs: Bool) -> Bool:
        return __mlir_op.`pop.and`(self._value, rhs._value)

    # if a == b: ...
    def __eq__(self, rhs: Bool) -> Bool:
        return __mlir_op.`pop.cmp`[
            pred=__mlir_attr.`#pop<cmp_pred eq>`
        ](self._value, rhs._value)
```

MLIR is key to accessing heterogeneous accelerators

```mojo
# condensed from max/kernels/src/structured_kernels/amd_tile_io.mojo

comptime lds_scope = __mlir_attr.`[#llvm.alias_scope<id = "amdgpu.AsyncCopies", ...>]`

@inline(.always)
def ds_read_tr16_b64_row(
    tile: TileTensor[_, _, address_space=.SHARED, ...],
) -> SIMD[tile.dtype, 4]:
    var lds_ptr = __mlir_op.`builtin.unrealized_conversion_cast`[
        _type=__mlir_type.`!llvm.ptr<3>`
    ](tile.ptr + ...)
    var raw = __mlir_op.`rocdl.ds.read.tr16.b64`[
        _type=__mlir_type.`vector<4 x bf16>`,
        noalias_scopes=lds_scope,
    ](lds_ptr)
    return ... # cast raw back to SIMD[tile.dtype, 4]
```

attribute · scheduling metadata as a comptime value

type · a pointer in LDS: address space 3

op · AMD's transposed LDS read

AI hardware compilers have MLIR dialects.

Add support for new hardware as a **Mojo library** problem with a small compiler plugin, not a new compiler fork.

The next compressed floating-point format needs **no compiler change**.

"Deferred" super-charges MLIR building from comptime parameters

```mojo
from std.collections.string.string_span import (
    _get_kgen_string,
)

# Deferred type: an n x n f32 vector tile
comptime Tile[n: Int] = __mlir_deferred_type[
    `vector<`, +n.__mlir_index__(), `x`,
    +n.__mlir_index__(), `xf32>`,
]

# Deferred attr: Mosaic matmul precision
def prec[p: StaticString]() -> __mlir_type.`!kgen.deferred`:
    return __mlir_deferred_attr[
        `#tpu.contract_precision<`,
        +_get_kgen_string[p](), `>`,
    ]

# Deferred op: tpu.matmul, verified once n binds
def matmul[n: Int, p: StaticString](
    x: Tile[n], y: Tile[n], acc: Tile[n]
) -> Tile[n]:
    return __mlir_op.`tpu.matmul`[
        _type=Tile[n],
        transpose_lhs_hint=__mlir_attr.false,
        precision=prec[p](),
    ](x, y, acc)

# matmul[128, "bf16"] →
#   tpu.matmul %x, %y, %acc
#     {precision = #tpu.contract_precision<bf16>}
#     : vector<128x128xf32>, ... -> vector<128x128xf32>
```

deferred type

deferred attributes

deferred op

New hardware from library code

Mojo compiler doesn't need to understand all vendor dialects

Generic code over any dialect

One definition covers every shape, dtype or mode

Checked against the real types

Elaboration turns deferred into concrete MLIR and verifies

IR you can read

11:30 – 12:00pm How Mojo 🔥 supports non-LLVM back-ends (TPU & Trainium) Kolya Panchenko

Domain-specific power tools are now libraries

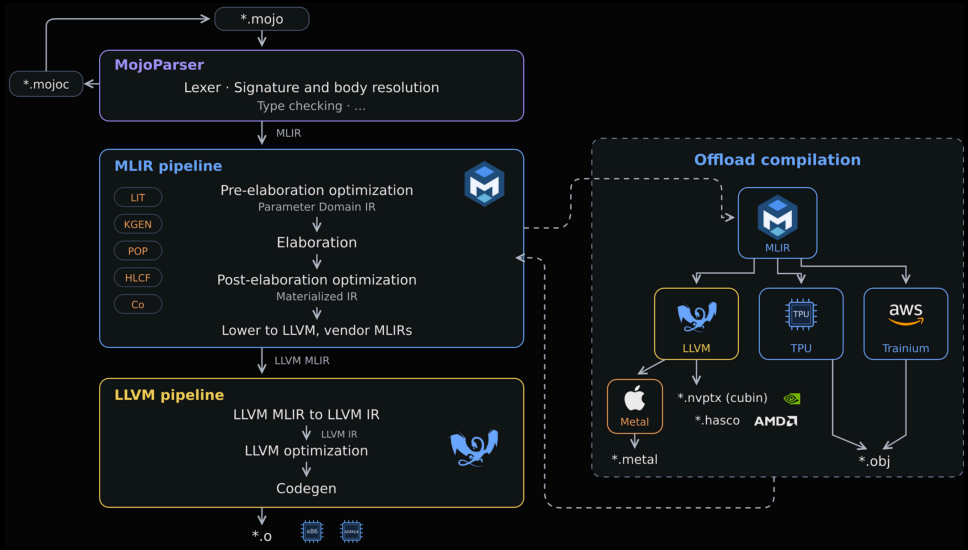
Made possible with zero-cost abstractions and a strong type system

```
# map one warp (32 threads) onto a 16×64 output tile
comptime thread_layout = TileTensor.row_major(8, 4)

# each thread's slice of the tile, resolved in the compiler
var elements = (
    output_tile
    .vectorize[1, 2]()
    .distribute[thread_layout](lane_id())
)

comptime for i in range(elements.shape[0]()):
    comptime for j in range(elements.shape[1]()):
        elements[i, j] = accumulator[i, j]
```

`TileTensor`, `vectorize`, `distribute`: van de Geijn + CuTe layout algebra, implemented as an ordinary Mojo library. It runs in the compiler's interpreter and costs nothing at runtime.

Heterogeneous IR in one Mojo program

```mojo
from std.math import ceildiv
from std.sys import has_accelerator

from layout import TileTensor, row_major
from max.gpu import global_idx
from max.gpu.host import DeviceContext

comptime dtype = DType.float32
comptime WIDTH = 10
comptime BLOCK = 8
comptime layout = row_major[WIDTH]()

def vector_addition(                                    # GPU kernel
    lhs: TileTensor[dtype, type_of(layout), ImmutAnyOrigin],
    rhs: TileTensor[dtype, type_of(layout), ImmutAnyOrigin],
    output: TileTensor[dtype, type_of(layout), MutAnyOrigin],
    size: Int32,
):
    var tid = global_idx.x
    if tid < Int(size):
        output[tid] = lhs[tid] + rhs[tid]

def main() raises:                                      # CPU host code
    comptime assert has_accelerator(), "needs a GPU"
    var ctx = DeviceContext()

    var lhs_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    var rhs_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    var out_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    lhs_buf.enqueue_fill(1.0)
    rhs_buf.enqueue_fill(2.0)

    var lhs = TileTensor(lhs_buf, layout)
    var rhs = TileTensor(rhs_buf, layout)
    var output = TileTensor(out_buf, layout)
    ctx.enqueue_function[vector_addition](
        lhs.as_imm(),
        rhs.as_imm(),
        output,
        Int32(WIDTH),
        grid_dim=ceildiv(WIDTH, BLOCK),
        block_dim=BLOCK,
    )

    with out_buf.map_to_host() as host_buf:
        print(TileTensor(host_buf, layout))
```



1. **Slice** device code into its own MLIR module and compile to binary
2. **Merge** compiled offload binary back to host module as final output

not a frontend split, not a runtime decision, not a second compiler invocation.

Mojo is open-source!

Contributing to Mojo

Mojo is fully open source on GitHub, and we're accepting contributions in several areas.

01
Compiler

02
Standard library

03
Docs

Each team accepts a specific set of changes, so check the contribution areas before you start work.

`github.com/modular/modular`

Mojo 🔥 Workshop Agenda

8:40 – 9:30am	Mojo 🔥 in the LLVM Ecosystem	Billy Zhu, Lukas Hermann, and Weiwei Chen
9:30 – 10:00am	Mojo Compiler Architecture Overview	Billy Zhu and Weiwei Chen
10:00 – 10:30am	Break	
10:30 – 11:00am	Mojo stdlib and driver plugins	Connor Gray and Manoj Rajagopalan
11:00 – 11:30am	Value Ownership, Lifetimes, and Origins: Mojo's Memory Safety Model	Chris Lattner
11:30 – 12:00pm	How Mojo 🔥 supports non-LLVM backends	Kolya Panchenko
12:00 – 12:30pm	MAX graph compiler & MLIR dialect for kernel fusion	Feras Boulala
12:30 – 1:30pm	Lunch	
1:30 – 2:00pm	How the Mojo standard library provides a nice interface over MLIR constructs	Joe Loser and Nathan Ward
2:00 – 2:30pm	Case Study: How Mojo Metaprogramming abstracts away hardware specificity	Kern Handa
2:30 – 3:00pm	Break	
3:00 – 3:30pm	Mojo Roadmap	Denis Gurchenkov
3:30 – 4:00pm	Modular Open Source Contribution Strategy	Brad Larson and Joe Loser
4:00 – 4:30pm	Q&A	
4:30 – 5:00pm	Mojo and the LLVM community Roundtables	